EEE22301 Project 2

Project 2 consists of two programming problems.

- **Problem 1: Smart Factory**
  - Part 1: Conveyor Belt Sorter
  - Part 2: Machine Job Scheduler
- **Problem 2: Matrix Equation Solver**

Please read the following instructions carefully and make sure to follow all coding and submission requirements.

📌 Submission Details

- **Deadline:** June 19, 2026 (23:59)
- **Late submissions will incur the following penalties:**
  - 1 day late: **20% deduction** (by June 20, 23:59)
  - 2 days late: **30% deduction** (by June 21, 23:59)
  - More than 2 days late: **Not accepted**
- Submit a ZIP file named `20250000.zip` using your student ID as the file name.
- The ZIP file must contain **only** the following files:

```
20250000.zip
├── 20250000_PJT2_P1.cpp
├── 20250000_PJT2_P2.cpp
├── problem1.txt
├── matrix2.txt
├── matrix3.txt
├── matrix4.txt
└── Assignment2.pdf
```

File	Description
`20250000_PJT2_P1.cpp`	Source code for Problem 1: Smart Factory
`20250000_PJT2_P2.cpp`	Source code for Problem 2: Matrix Equation Solver
`problem1.txt`	Generated output file for Problem 1 Part 1 and Part 2
`matrix2.txt`	Input file for the 2×2 matrix equation
`matrix3.txt`	Input file for the 3×3 matrix equation
`matrix4.txt`	Input file for the 4×4 matrix equation
`Assignment2.pdf`	Report for LLM usage and code understanding

> ⚠️ **Naming rule violations will result in a 5-point deduction per problem.**

⚠️ Additional Submission Warnings

- Submit exactly one ZIP file named `20250000.zip`.
  - Incorrect examples: `20250000.zip.zip`, `20250000.zip (2)`, `2025000.zip`
- The ZIP file must contain only the required files listed above.
  - Incorrect examples: `__MACOSX/`, `.vs/`, extra folders, backup files, or unnecessary files
- Submissions that include incorrectly named ZIP files, duplicate ZIP extensions, or unnecessary files/folders will be penalized.

📌 General Coding Rules

- Each program must:
  - Use `cin` for input.
  - Use `cout` for output.
  - Compile and run correctly.
  - Produce output in the exact required format.
  - Not print unnecessary text unless the problem explicitly requires it.
  - Preserve the provided function structure.
  - Modify only the TODO sections.
- Compile errors will result in **0 points** for the corresponding problem.
- Follow the allowed library rules specified separately for each problem.
- All required output formats must be followed exactly.
- Include at least **two meaningful `//` comments explaining the program logic.** The comments may be placed anywhere in the source code. This requirement applies separately to `20250000_PJT2_P1.cpp`, `20250000_PJT2_P2.cpp`.
  - All comments must be written in English.
  - Simple comments such as `// loop` or `// print` are not considered meaningful comments.
  - Example of meaningful comments

```cpp
#include <iostream>
using namespace std;

int add(int a, int b) {
    // Calculate the sum of two input numbers
    return a + b;
}

void printResult(int value) {
    // Print final result to the screen
    cout << value << endl;
}

int main() {
    int x = 3;
    int y = 5;

    int result = add(x, y);
```

```
        printResult(result);

        return 0;
    }
```

📌 AI Tool Policy

- The use of LLMs or AI tools is allowed for both **Problem 1** and **Problem 2**.
- However, students are fully responsible for the correctness and understanding of their submitted code.
- Students must submit `Assignment2.pdf`, which includes the required **LLM Usage** section.
- The report is **NOT** intended to collect full AI chat logs. It is intended to verify that students understand their own code, can explain the algorithms, and can describe how they debugged, modified, and verified their implementation.

Additional Bonus Policy

- Students who implement the solution without using LLMs or AI tools may receive up to **10 bonus points per problem** (up to 20 bonus points total).
- The bonus is awarded based on demonstrated code understanding, implementation process, and debugging explanation.
- The report must clearly show the student's own reasoning and development process.
- The TA may ask additional questions or request clarification if necessary.
- False claims regarding AI tool usage may result in a score of 0 for the corresponding problem after verification.

📌 Grading Criteria

The total score is **100 points**. Bonus points may allow the final score to exceed 100 points.

- **Code test cases:** 40 pts
- **Report (`Assignment2.pdf`):** 60 pts

Component	Code Test Points	Report Points	Total
Problem 1 - Part 1: Conveyor Belt Sorter	8	18	26
Problem 1 - Part 2: Machine Job Scheduler	8	18	26
Problem 2: Matrix Equation Solver	24	24	48
Total	**40**	**60**	**100**

Code Test Case Breakdown

- Problem 1 - Part 1:
    - 4 seed test cases = **8 pts**
- Problem 1 - Part 2:

    - 4 seed test cases = **8 pts**

- Problem 2:

    - `matrix2.txt`, `matrix3.txt`, and `matrix4.txt` = **24 pts**
    - Each matrix file is worth **8 pts**

📌 Report Requirement: LLM Usage and Code Understanding

Students must submit a report file named `Assignment2.pdf`. The report must be no longer than **6 pages in total, including the cover page**. Only the first 6 pages may be graded; content beyond the page limit may not be considered.

The purpose of this report is not to collect full AI chat logs. The purpose is to verify that students understand their own code, can explain the algorithms, and can describe how they debugged, modified, and verified their implementation.

Use of LLMs or AI tools is allowed. However, students are fully responsible for the correctness and understanding of their submitted code.

Required Structure

Students must write the following report sections **separately for each component**:

1. **Problem 1 - Part 1: Conveyor Belt Sorter**
2. **Problem 1 - Part 2: Machine Job Scheduler**
3. **Problem 2: Matrix Equation Solver**

For each of the three components above, students must answer all of the following items.

1. Problem Encountered or Difficult Part (Assignment2_Template - A section)

Describe one issue encountered while implementing this component.

If there was no clear bug or error, describe the part that was most difficult to understand or implement.

Do not simply write that there was no problem.
Even if the implementation worked successfully, explain which part required the most careful reasoning.

2. How You Solved It (Assignment2_Template - B,C section)

Explain how the issue or difficult part was resolved. Choose ONE of the following cases.

1) If LLM was used

Include the following:

- A screenshot of the code **before** the change
    - A short explanation of what the problem is
- A short summary of the prompt in your own words
- A short summary of the LLM response in your own words

- A screenshot of the code **after** the change
  - A short explanation of what changed
- An explanation of how the AI response was applied to the submitted code
- An explanation of how you verified that this part of the code works correctly

Students should not submit a long raw chat log.
The report should summarize the important reasoning and decisions.

The report must make clear that the student understood the code change rather than blindly copying an AI-generated answer.

2) If LLM was not used

Include the following:

- A screenshot of the code **before** the change
  - A short explanation of what the problem is
- A description of the debugging or problem-solving process
- A screenshot of the code **after** the change
  - A short explanation of what changed
- An explanation of how you verified that this part of the code works correctly

3. Code Logic Explanation

Students must explain the main logic of their implementation using one given test case or input example.

The purpose of this section is to show that students understand how their code produces the result, not to provide a full manual calculation.

Problem 1 - Part 1: Conveyor Belt Sorter

Explain how your code computes the minimum number of adjacent swaps.

The explanation must include:

- the main logic used to compute the minimum number of adjacent swaps, including the use of inversion counting
- why the inversion count represents the minimum number of adjacent swaps
- how the output value `K` is obtained from the given shuffled sequence

Problem 1 - Part 2: Machine Job Scheduler

When `seed = 666`, explain the answer to the following Part 2 question:

```
Q. What is the maximum number of non-overlapping jobs with start_time >= K?
```

The explanation must include:

- the value of `K`

- the value of `R`
- which jobs are selected
- the logical process of how the selected jobs are determined

Problem 2

Using the 4×4 matrix in `matrix4.txt`, explain how the matrix equation is solved.

The explanation must include:

- the logical process used to obtain the solution
  - How each pivot row is normalized
  - How non-pivot rows are eliminated using row operations
- the final matrix (shown from the program output)

📌 Contact

For any questions, contact the TA.

- **k1d2m3@unist.ac.kr**
- **kim24@unist.ac.kr**

Problem 1: Smart Factory

This problem consists of two connected parts: **Part 1** and **Part 2**.
The answer from Part 1 is directly used as input to Part 2.

```
seed (input)
    │
    ▼
[Part 1]  Minimum adjacent swaps to sort → K
    │
    ▼
[Part 2]  Maximum jobs schedulable from time K → answer
```

Problem 1 Coding Rules

- Allowed libraries: `<iostream>`, `<fstream>`, `<random>`, `<string>`
- `std::sort`, `std::shuffle`, and `std::random_shuffle` are **strictly prohibited**.
- All sorting must be implemented manually.
- Random number generation must use `std::mt19937` and `std::uniform_int_distribution<int>`.
- The same seed must always produce the same output.

- `rng` must be created once in `main()` and passed by reference to all required functions.
- Do **not** re-initialize `rng` inside any function.

Required Utilities from `<random>`

`std::mt19937`

A pseudo-random number generator based on the Mersenne Twister algorithm.
It produces the **same sequence** for a given seed every time.

`std::uniform_int_distribution<int>`

Selects a random integer uniformly within a specified range `[a, b]`.

```
std::mt19937 rng(seed);

std::uniform_int_distribution<int> d(1, 100);
int x = d(rng);
```

> **Important:** `rng` is created once in `main()` and passed by reference to all functions.
> Never re-initialize it inside a function. Doing so will break reproducibility.

References:

- https://cplusplus.com/reference/random/mt19937/
- https://cplusplus.com/reference/random/uniform_int_distribution/

Problem 1 Required Runs

The program reads the seed using `cin`.

When the console window is waiting for input, type one seed value and press Enter.

Example input:

```
7
```

The program will:

- print the generated Part 1 and Part 2 problem statements and answers to the console
- append the full Problem 1 result, including both Part 1 and Part 2, to `problem1.txt`

To generate all required results, run the program four times and enter the following seeds one by one:

Run	Seed
1	7
2	56
3	230
4	3

`problem1.txt` accumulates across runs.
If you want to start over, delete `problem1.txt` before running again.

Part 1: Conveyor Belt Sorter

Grading: 8 pts for code test cases + 18 pts for report

---

1) Problem Description

Parts arrive in a shuffled order.
The robot arm can only swap **two adjacent parts** at a time.

Find the **minimum number of adjacent swaps** required to sort the parts in ascending order.

This value is called **K** and is used directly in Part 2.

---

2) Console Input / Output

	Description
Input	A single integer `seed` where `0 ≤ seed ≤ 100000`
Output	Shuffle debugging output, then the problem statement and answer K

---

3) Example 1: seed = 40

```
[Initial array]:       6 12 79 71 29 15 46 59 31 14 53 18 63 92 78 11 69 51

Shuffled 7 times:
[After shuffle #1]:   53 12 78 11 29 46 92 59 79  6 71 69 15 18 63 14 31 51
...
[After shuffle #7]:   29 79 15 53 31 11 18  6 71 69 92 59 12 78 46 14 63 51

[Sorted (ascending)]:  6 11 12 14 15 18 29 31 46 51 53 59 63 69 71 78 79 92
```

```
Q. The following is a sequence of 18 parts arriving in shuffled order.
   What is the minimum number of adjacent swaps to sort the sequence in ascending
```

```
order?
   29 79 15 53 31 11 18 6 71 69 92 59 12 78 46 14 63 51
A. 72
```

---

4) Example 2: seed = 666

```
[Initial array]:       85 22 68 13 73 76 96 34  2 25 42 26  5 29 10 47 51 24 21 77
75 82 20 40

Shuffled 2 times:
[After shuffle #1]:   75 73 82 10 21  2 20 76 26 24 42 68 25 77 40 22  5 85 29 47
13 96 34 51
[After shuffle #2]:   75 85 42  2 26 40  5 82 51 25 77 10 13 20 22 47 24 73 21 29
96 68 34 76

[Sorted (ascending)]:  2  5 10 13 20 21 22 24 25 26 29 34 40 42 47 51 68 73 75 76
77 82 85 96
```

```
Q. The following is a sequence of 24 parts arriving in shuffled order.
   What is the minimum number of adjacent swaps to sort the sequence in ascending
order?
   75 85 42 2 26 40 5 82 51 25 77 10 13 20 22 47 24 73 21 29 96 68 34 76
A. 127
```

---

5) Function Specifications

(1) `generateWeights`

```
void generateWeights(int& N, int weights[], mt19937& rng);
```

Generate a shuffled sequence of part weights and print the shuffle process.

Requirements

Generate the following values using `rng`:

- `N`: number of parts, range `[10, 30]`
- `K_shuffle`: number of shuffle passes, range `[1, 10]`
- `N` weight values, each in range `[1, 100]`

Then shuffle the array `K_shuffle` times using **Fisher-Yates Shuffle**.

Shuffle Debugging Output Format

```
[Initial array]: 6 12 79 71 29 15 46 59 ...
[After shuffle #1]: 53 12 78 11 29 46 92 59 ...
[After shuffle #2]: 18 53 71 29 51 31 15 12 ...
...
[After shuffle #K_shuffle]: 29 79 15 53 31 11 18 6 ...
```

Fisher-Yates Shuffle

`std::shuffle` is **prohibited**. Implement the following manually:

```
For each shuffle pass:
  For i from N-1 down to 1:
    Generate a random index j in [0, i]
    Swap weights[i] and weights[j]
```

Use:

```
std::uniform_int_distribution<int> d(0, i);
```

Reference:

- https://en.wikipedia.org/wiki/Fisher%E2%80%93Yates_shuffle

---

(2) `mergeSortCount`

```
long long mergeSortCount(int arr[], int left, int right);
```

Sort `arr[left..right]` using Merge Sort and return the number of **inversion pairs**.

What is an Inversion?

A pair `(i, j)` is an inversion if:

```
i < j and arr[i] > arr[j]
```

The total number of inversions is equal to the minimum number of adjacent swaps required to sort the array.

Example:

```
arr = [3, 1, 2]

(3,1) → 3 > 1 and index 0 < index 1  inversion
(3,2) → 3 > 2 and index 0 < index 2  inversion
(1,2) → 1 < 2                         not an inversion

Total = 2  →  minimum adjacent swaps = 2
```

Duplicate Values

If `arr[i] == arr[j]`, it is **not** an inversion.

```
arr = [3, 3, 1]  →  inversions: (3,1), (3,1) = 2
```

Reference:

- https://www.geeksforgeeks.org/dsa/merge-sort/

---

(3) `countInversions`

```
long long countInversions(int weights[], int N);
```

Return the total inversion count of `weights` as **K**.

Requirements

- Do **not** modify the original `weights` array.
- Return type must be `long long`.

---

(4) `saveToFile`

```
void saveToFile(int seed, const string& content);
```

Append the full Problem 1 content to `problem1.txt`.

Requirements

- Open `"problem1.txt"` in **append mode**.
- Write the seed header, generated problem content, and separator.

Expected Format

```
== seed: 2 ==
Q. The following is a sequence of ...
A. 90

[Machine available from time K=90, rest time R=1]
Q. What is the maximum number of non-overlapping jobs ...
A. 4
=======================================
```

---

6) Notes and Edge Cases

Case	Expected Behavior
Already sorted array	K = 0
Fully reversed array of size N	K = N×(N−1)/2
Duplicate elements `[3, 3, 1]`	K = 2
Single element	K = 0

Part 2: Machine Job Scheduler

Grading: 8 pts for code test cases + 18 pts for report

---

1) Problem Description

The factory starts at **time 0**.
Sorting finishes at **time K**.

The machine is available from **time K onwards**.

After each job, the machine needs **R minutes of maintenance**:

```
R = K % 5 + 1
```

Find the **maximum number of non-overlapping jobs** the machine can process.

---

2) Timeline

```
Time 0                  Time K
  |                       |
  ├─── Sorting parts ─────┼──────── Machine available ──────▶
```

```
                          ↑
                  Machine starts here

  start < K  →  not eligible
  start ≥ K  →  eligible
```

---

3) Example 1: seed = 40

```
[Machine available from time K=72, rest time R=3]
Q. What is the maximum number of non-overlapping jobs with start_time >= K?
   Job   Start   End
   ---   -----   ---
   1     79      81
   2     85      90
   3     78      80
   4     96      105
   5     93      97
   6     68      78
   7     66      73
   8     94      97
   9     64      67
   10    89      91
A. 3
```

---

4) Example 2: seed = 666

```
[Machine available from time K=127, rest time R=3]
Q. What is the maximum number of non-overlapping jobs with start_time >= K?
   Job   Start   End
   ---   -----   ---
   1     136     138
   2     144     152
   3     130     133
   4     128     130
   5     119     125
   6     140     143
   7     152     161
   8     150     153
   9     132     136
   10    132     139
A. 3
```

---

5) Function Specifications

(5) `generateJobs`

```
void generateJobs(long long K, int& M, int& R, Job jobs[], mt19937& rng);
```

Generate jobs using the **same** `rng` from Part 1.

Do **not** re-initialize `rng`.

Requirements

- `R = (int)(K % 5) + 1`
- `M`: number of jobs, range `[8, 15]`
- Each job:
  - `start` in range `[max(0, K-8), K+25]`
  - `duration` in range `[2, 10]`
  - `end = start + duration`
  - `id = i + 1`

---

(6) `maxNonOverlappingJobs`

```
int maxNonOverlappingJobs(long long K, int R, Job jobs[], int M);
```

Return the maximum number of jobs the machine can process.

Selection Conditions

```
A job is eligible if:        job.start >= K
A job can follow another if: next.start >= last_selected.end + R
```

---

6) Notes and Edge Cases

Case	Expected Behavior
No eligible jobs after filtering	Return 0

---

7) Test Case Summary

seed	N	K	R
7	11	28	4

seed	N	K	R
56	30	269	5
230	23	120	1
3	21	94	5

Problem 2: Matrix Equation Solver

Write a program that reads a matrix equation from a text file and solves it using row reduction.

The equation has the following form:

```
Ax = b
```

Instead of storing `A` and `b` separately, the file stores the equation as an augmented matrix:

```
[A | b]
```

Your program must read the augmented matrix from a file, reduce the left side to the identity matrix, and print the solution vector `x`.

The input file contains a 2×2, 3×3, or 4×4 matrix equation.

Problem 2 Coding Rules

- Allowed libraries: `<iostream>`, `<fstream>`, `<string>`
- `<vector>`, `<algorithm>`, `<cmath>`,and `<iomanip>` are **strictly prohibited**.
- Do not modify the provided `main()` function.
- Use the given function structure.
- Modify only the TODO sections.

---

1) Problem Description

In this problem, you will implement a program that solves matrix equations using row reduction.

Each file may contain:

- 2×2 matrix equation
- 3×3 matrix equation
- 4×4 matrix equation

Your program must:

1. Ask the user to enter a filename.
2. Open the file using `ifstream`.
3. Read the matrix size `n`.
4. Read the augmented matrix `[A | b]`.
5. Normalize each pivot row.
6. Eliminate all other values in the pivot column.
7. Print the intermediate matrix, after each pivot step.
8. Print the final solution from the last column.

All test cases are guaranteed to:

- have a unique solution
- have integer solutions
- not require row swapping

---

2) Matrix File Format

The first line of the file contains one integer `n`.

The next `n` lines contain `n + 1` integers.

Each line represents one equation:

```
a1 a2 ... an b
```

This means:

$a_1x_1 + a_2x_2 + ... + a_nx_n = b$

In the program output, variables are printed using zero-based indexing: `x[0]`, `x[1]`, ..., `x[n-1]`.

---

3) Console Input / Output

Input

The program first asks for a filename:

```
Enter the filename:
```

Example:

```
Enter the filename: matrix3.txt
```

Output

Your program must print the matrix after each pivot step, followed by the final solution.

Intermediate Matrix Output Format

```
After Pivot 0
...
```

Each row of the matrix should be printed on one line . Values in the same row should be separated by a single space.

For a 3×3 matrix equation, each row contains 4 values because the last column is b.

Final Solution Format

```
x[0] = ...
x[1] = ...
x[2] = ...
```

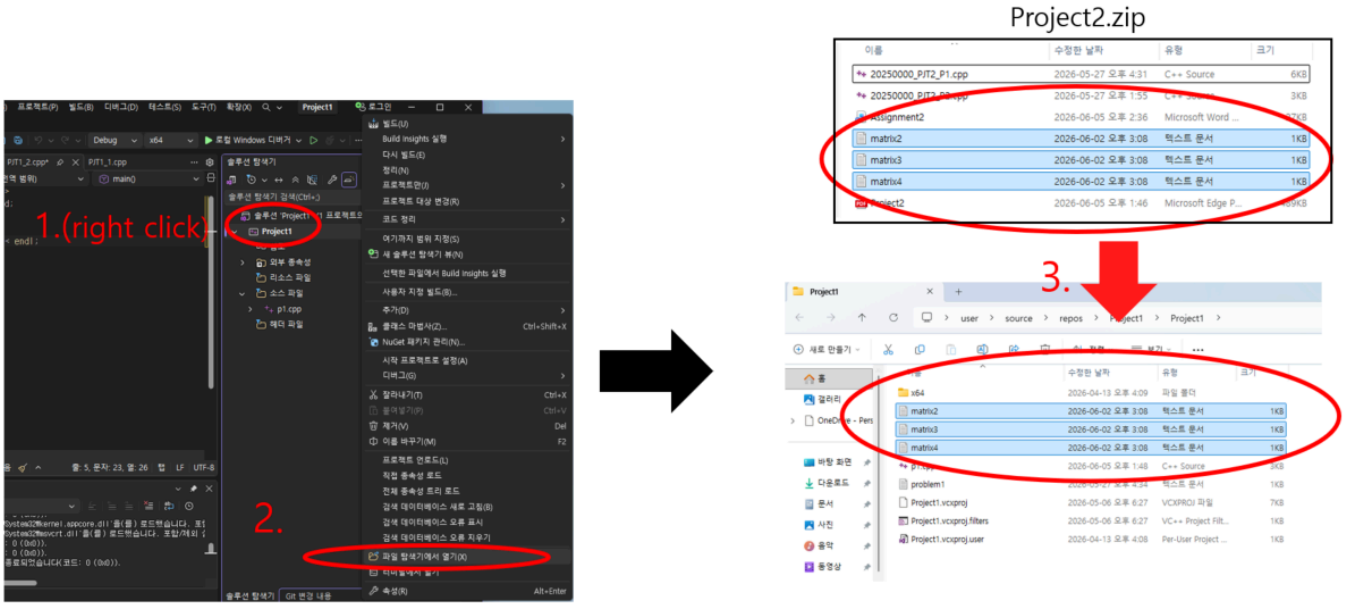
Visual Studio File Placement

If you are using Visual Studio:

1. Copy the provided test files (matrix2.txt, matrix3.txt, and matrix4.txt) into the project folder.
2. Run the program and enter the filename when prompted.

Steps:

1. Right-click the project in Visual Studio.
2. Select "Open Folder in File Explorer(파일 탐색기에서 열기)".
3. Copy matrix2.txt, matrix3.txt, and matrix4.txt into the project folder.



4) Row Reduction Example

Suppose the augmented matrix is:

```
[ 1  1  1 |   5 ]
[ 2  3  5 |   8 ]
[ 4  0  5 |   2 ]
```

Using row reduction, the matrix can be transformed as follows:

```
Pivot 0:
R2 = R2 - 2R1
R3 = R3 - 4R1

[ 1  1  1 |   5 ]
[ 0  1  3 |  -2 ]
[ 0 -4  1 | -18 ]
```

```
Pivot 1:
R1 = R1 - R2
R3 = R3 + 4R2

[ 1  0 -2 |   7 ]
[ 0  1  3 |  -2 ]
[ 0  0 13 | -26 ]
```

```
Pivot 2:
R3 = (1/13)R3
R1 = R1 + 2R3
R2 = R2 - 3R3

[ 1  0  0 |   3 ]
[ 0  1  0 |   4 ]
[ 0  0  1 |  -2 ]
```

Therefore, the solution vector is:

```
x[0] = 3
x[1] = 4
x[2] = -2
```

---

5) Examples

Example 1: 2×2 Matrix

matrix2.txt

```
2
1 2 5
3 4 11
```

This represents:

$x_1 + 2x_2 = 5$
$3x_1 + 4x_2 = 11$

Output

```
After Pivot 0
1 2 5
0 -2 -4

After Pivot 1
1 0 1
0 1 2

x[0] = 1
x[1] = 2
```

Example 2: 3×3 Matrix

matrix3.txt

```
3
1 1 1 5
2 3 5 8
4 0 5 2
```

This represents:

$x_1 + x_2 + x_3 = 5$
$2x_1 + 3x_2 + 5x_3 = 8$
$4x_1 + 5x_3 = 2$

Output

```
After Pivot 0
1 1 1 5
0 1 3 -2
0 -4 1 -18

After Pivot 1
1 0 -2 7
0 1 3 -2
0 0 13 -26

After Pivot 2
1 0 0 3
0 1 0 4
0 0 1 -2

x[0] = 3
x[1] = 4
x[2] = -2
```

---

6) Constraints

```
2 ≤ n ≤ 4
```

You may assume that:

- the file exists
- the file format is valid
- pivot values are never zero
- all solutions are guaranteed to be integers
- all intermediate matrix values printed,after each pivot step are integers

---

7) Function Specifications

(1) `readAugmentedMatrix`

```
void readAugmentedMatrix(ifstream& inputFile, int& n, double matrix[4][5]);
```

This function reads the augmented matrix from a file.

Requirements

1. Read the value of `n`.
2. Read `n` rows and `n + 1` columns into `matrix`.

---

(2) `normalizePivotRow`

```
void normalizePivotRow(int n, double matrix[4][5], int pivot);
```

This function finds the pivot value in the current pivot row Then, it divides every element in the row by the pivot value. Example:

```
2 4 -2 | 8
```

becomes:

```
1 2 -1 | 4
```

Requirements

1. Store the pivot value:.`matrix[pivot][pivot]`.
2. Divide every value in the pivot row by the pivot value
3. Include the last column as well.

---

(3) `eliminateColumn`

```
void eliminateColumn(int n, double matrix[4][5], int pivot);
```

This function makes every other value in the pivot column zero.

Requirements

For every row except the pivot row:

1. Get the factor from `matrix[row][pivot]`.
2. Subtract `factor × pivot row` from the current row.
3. Include the last column as well.

---

(4) `reduceToIdentity`

```
void reduceToIdentity(int n, double matrix[4][5]);
```

This function performs row reduction step by step. It repeats the process until the left side becomes the identity matrix.

Requirements

For each pivot index from `0` to `n - 1`:

```
normalizePivotRow(n, matrix, pivot);
eliminateColumn(n, matrix, pivot);
```

After each pivot step, print the current augmented matrix directly inside this function.

You must not create a separate matrix-printing function.

The output format must be:

```
After Pivot 0
...
```

Add one blank line after each pivot step except after the last pivot step.

---

(5) `printSolution`

```
void printSolution(int n, double matrix[4][5]);
```

Print the solution from the last column of the reduced matrix.

Since all final solution values are guaranteed to be integers, print them as integers.

Output format:

```
x[0] = ...
x[1] = ...
x[2] = ...
```